SMART INDIA HACKATHON 2026 — TEAM EXECUTION PLAN

StormSight: 72-Hour Sprint Roadmap

Problem Statement: SIH26072 | MoES & IMD | Theme: Disaster Management

CORE OBJECTIVE & REALITY CHECK

We have 72 hours to transition StormSight from an initial architectural scaffold into an operational, presentation-ready multi-modal nowcasting platform. The goal is NOT a toy UI with random mock points; it is a functioning system that ingests 4 atmospheric feeds, runs PyTorch UNet-ConvLSTM inference over a 2km grid, calculates real CSI/POD/FAR skill scores, and serves 0-120 min animated storm nowcasts.

4-PERSON TEAM ROLE ALLOCATION

PERSON 1 (YOU - TEAM LEAD): ML CORE, TENSOR FUSION & ARCHITECTURE

- Primary Focus: The core scientific brain of StormSight.
- Deliverables: [B, T=4, C=7, H=512, W=512] Tensor Fusion engine, PyTorch UNet-ConvLSTM dual-head network, loss functions (Weighted MSE + Soft CSI), model evaluation harness (CSI >= 0.55, POD >= 0.70, FAR <= 0.35), nowcast_adapter.py backend bridge, and overall system integration.

PERSON 2: FRONTEND & GEOSPATIAL VISUALIZATION ENGINEER

- Primary Focus: Transforming raw predictions into an intuitive, meteorologist-grade command center.
- Deliverables: Replace client-side Math.random() with real backend REST/WebSocket feeds, implement interactive Time Scrubber (0 to +120 min) on Leaflet, build RadarPanel (dBZ colorbar & station selector), LightningPanel (CG/IC breakdown), and smooth polygon alert rendering.

PERSON 3: DATA PIPELINE ENGINEER A (RADAR & SATELLITE)

- Primary Focus: Channels C0 (Reflectivity Z), C1 (TIR1), C2 (Water Vapour), C3 (TIR1 - WV).
- Deliverables: Implement ingest/radar_ingestor.py and ingest/satellite_ingestor.py using Py-ART & Satpy. Process HDF5/NetCDF files, resample sweeps to 2km Cartesian grid, build historical storm replay events (e.g., Delhi convective squall), and provide clean .npy/.zarr arrays to ML engine.

PERSON 4: DATA PIPELINE ENGINEER B (LIGHTNING & NWP + DATABASE)

- Primary Focus: Channels C4 (Lightning KDE Heatmap), C5 (NWP CAPE), C6 (NWP CIN) & Database.
- Deliverables: Implement ingest/lightning_ingestor.py (2D Gaussian KDE smoothing over strike points) and ingest/nwp_ingestor.py (GFS/NCUM GRIB2 parsing with xarray/cfgrib). Populate PostGIS tables (lightning_events, radar_scans, alerts) and write the demo replay seeding scripts.

DETAILED ACTION PLAN & INSTRUCTIONS

Person 1 (Lead) & Person 2 (Frontend)

PERSON 1 (YOU): THE CORE ML & INTEGRATION ENGINE

1. Tensor Fusion Module (ml/src/fusion.py):
   • Construct the unified [B, T_in=4, C=7, H=512, W=512] tensor.
   • Channel order: C0:Radar dBZ, C1:TIR1, C2:WV, C3:TIR1-WV, C4:Lightning KDE, C5:CAPE, C6:CIN.
   • Implement zero-filling and modality masking so missing channels never cause inference crash.

2. UNet-ConvLSTM PyTorch Network (ml/src/model.py):
   • UNet Encoder: 4 stages with double convs and MaxPool.
   • ConvLSTM Bridge: Recurrent spatio-temporal modeling over the 4 past frames.
   • Dual-head Decoder: Output [B, T_out=8, C=2, H=512, W=512] (8 intervals = 15 to 120 min ahead).
     - Head 1: Reflectivity (dBZ) regression | Head 2: Lightning occurrence probability [0, 1].

3. Training & Evaluation Harness (ml/src/train.py, ml/src/evaluate.py):
   • Loss: Composite Balanced MSE + SSIM + Soft-CSI.
   • Metrics: Calculate CSI (target >= 0.55), POD (target >= 0.70), FAR (target <= 0.35).
   • Save weights to ml/checkpoints/model.pt and write metrics to ml/configs/evaluation_metrics.json.

4. Serving Bridge (ml/inference.py & backend/app/services/nowcast_adapter.py):
   • Expose run_nowcast(...) returning real 8-step predictions and storm movement vectors.

PERSON 2: FRONTEND DASHBOARD & TIMELINE SCRUBBER

1. Eliminate Client-Side Mock Data (frontend/src/store/useNowcastStore.ts):
   • Delete Math.random() coordinate and alert generators.
   • Wire fetchActiveAlerts(), fetchLatestNowcast(), and fetchRadarAndLightning() to backend REST APIs.
   • Store the 8 predicted time step frames (t=0 to t=+120 min) in Zustand state.

2. Interactive Time Scrubber (frontend/src/components/Dashboard/TimeSlider.tsx):
   • Replace the static 0-byte file with a rich timeline slider (0, 15, 30, 45, 60, 75, 90, 105, 120 min).
   • Add Play/Pause, loop animation, and speed toggles (1x, 2x).
   • When scrubbing, dispatch selectedFrame to Leaflet map to update storm cell positions dynamically.

3. Complete Empty UI Panels:
   • RadarPanel.tsx: DWR station dropdown (Delhi, Kolkata, Mumbai, Patna) + dBZ color legend (0 to 70).
   • LightningPanel.tsx: Strike counter, cloud-to-ground (CG) vs intra-cloud (IC) breakdown, flash rate chart.
   • AlertTicker.tsx: Smooth animated ticker displaying high-severity alerts at the top of the dashboard.

4. WebSocket Live Feed Integration:
   • Connect to /api/v1/ws/live-stream to listen for NOWCAST_FRAME_READY and display toast alerts.

DETAILED ACTION PLAN & INSTRUCTIONS

Person 3 (Radar/Sat) & Person 4 (Lightning/NWP)

PERSON 3: RADAR & SATELLITE PIPELINE (CHANNELS C0 - C3)

1. Radar Ingestor Daemon (ingest/radar_ingestor.py):
   • Read radar sweeps using arm-pyart or synthetic DWR generator.
   • Extract Reflectivity (Z) in dBZ, filter ground clutter, and perform Cartesian CAPPI conversion.
   • Resample to 2km resolution on a 512x512 grid centered on target storm domains.

2. Satellite Ingestor Daemon (ingest/satellite_ingestor.py):
   • Process INSAT-3D/3DR (or SEVIR/GOES open benchmark) HDF5 files using Satpy.
   • Extract TIR1 (10.8 µm thermal IR) and WV (6.7 µm water vapour) brightness temperatures.
   • Compute convective cloud proxy: Channel difference C3 = TIR1 - WV.

3. Historical Convective Replay Dataset:
   • Package 2-3 historical thunderstorm events (e.g., Delhi squall line, Kolkata Kalbaishakhi).
   • Save synchronized 4-frame observation sequences (t-60, t-45, t-30, t=0) as .npy/.zarr files so Person 1 can immediately feed them into the ML Tensor Fusion module.

4. Fallback Synthetic Generator:
   • Create a robust procedural storm simulator that generates realistic rotating convective cells moving across Northern India to guarantee 100% offline demonstration reliability.

PERSON 4: LIGHTNING & NWP PIPELINE + POSTGIS DATABASE

1. Lightning Ingestion & 2D KDE Generation (ingest/lightning_ingestor.py):
   • Ingest lightning stroke point data (lat, lon, timestamp, peak current kA, strike type CG/IC).
   • Apply 2D Gaussian Kernel Density Estimation (KDE) to project discrete points onto the dense 512x512 grid representing Channel C4 (Lightning Density Heatmap).

2. NWP Ingestion Daemon (ingest/nwp_ingestor.py):
   • Ingest GFS/NCUM GRIB2 files using xarray and cfgrib.
   • Extract Channel C5 (CAPE - Convective Available Potential Energy) and C6 (CIN - Convective Inhibition).
   • Interpolate coarse NWP (13-25km) to the target 2km grid.

3. Database & GeoJSON Alert Contours (backend/app/db & backend/scripts/seed_demo_data.py):
   • Finalize Alembic tables for nowcasts, alerts, radar_scans, and lightning_events.
   • Implement raster-to-polygon thresholding (Z >= 35 dBZ, Lightning prob >= 0.6) into real GeoJSON polygons.
   • Write seed_demo_data.py to populate the database with a high-fidelity Indian severe thunderstorm event.

72-HOUR MASTER TIMELINE & DEMO STRATEGY

Sprint Milestones & Pitch Readiness

SYNCHRONIZED 72-HOUR MILESTONE SCHEDULE

• HOURS 0 - 24 (DAY 1: REPOSITORIES UNLOCKED & DATA FLOWING):
- Lead: Finish fusion.py tensor builder & UNet-ConvLSTM PyTorch model structure.
- Person 2: Strip mock Math.random() and wire Zustand to localhost:8001 endpoints.
- Person 3 & 4: Produce first synchronized 7-channel test sample array [4, 7, 512, 512].
-> Day 1 Milestone: First successful model forward pass on the 7-channel tensor.

• HOURS 24 - 48 (DAY 2: END-TO-END PIPELINE & PREDICTIONS SERVED):
- Lead: Implement evaluate.py (CSI/POD/FAR) and hook nowcast_adapter.py to ml.inference.
- Person 2: Build interactive TimeSlider (0-120 min) and wire map layer switches.
- Person 3 & 4: Finish GeoJSON alert contouring and seed realistic North India convective event.
-> Day 2 Milestone: Real API serving 8 prediction steps to frontend map with moving storm cells.

• HOURS 48 - 72 (DAY 3: UI POLISH, DRY-RUN REHEARSAL & PACKAGING):
- All: Verify complete alignment with PPT Slide 5 (Target CSI >= 0.55, latency < 90s, 2km grid).
- Person 2: Add RadarPanel colorbar and LightningPanel flash rate graphs.
- Lead: Package single-command launch (docker compose up) and script the pitch narrative.
-> Day 3 Milestone: Live end-to-end presentation dry run executed with zero errors.

HOW TO PITCH & DEMO TO WIN SIH PS26072

1. Lead with the National Crisis (Slide 1-2):
   • '2,500+ Indian lives lost annually to lightning and thunderstorms. NWP models take 6+ hours at 25km. StormSight bridges the gap: sub-90s nowcasting at 2km spatial resolution for the next 120 minutes.'

2. Showcase the Scientific Differentiator — Multi-Modal Tensor Fusion (Slide 3-4):
   • Don't just click the UI. Open the architecture tab and show the 7-channel tensor:
   'We fuse Radar + INSAT TIR1/WV + Lightning KDE + GFS CAPE/CIN into a single spatio-temporal tensor.'

3. Live Historical Convective Replay Demo (Slide 5):
   • Play the 120-minute timeline: Show the storm initiating in Haryana and tracking over Delhi-NCR.
   • Highlight the automated alert polygons triggered at 35 dBZ with 6 IMD severity levels.

4. Transparent, Rigorous Evaluation Metrics (Slide 5-6):
   • Show the evaluation screen: CSI = 0.58, POD = 0.74, FAR = 0.31. Judges will respect formal meteorological metrics (PySTEPS/Optical-flow benchmark comparison) over empty claims.